MODUL VIII
OTOMASI CI/CD PIPELINE DAN PEMAKETAN APLIKASI

I. Tujuan Praktikum

1. Mahasiswa memahami konsep dasar DevOps serta penerapan Continuous Integration (CI) untuk peningkatan kualitas perangkat lunak secara berkelanjutan.
2. Mahasiswa mampu mendokumentasikan dependensi proyek menggunakan requirements.txt dan mengonfigurasi pipeline CI otomatis menggunakan GitHub Actions.
3. Mahasiswa mampu mengemas (packaging) seluruh kode sumber aplikasi desktop Python MVC menjadi berkas eksekusi mandiri (.exe) yang siap didistribusikan kepada pengguna akhir.

II. Alat dan Bahan

1. PC/Laptop
2. Visual Studio Code
3. Git dan Akun GitHub
4. Laragon
5. Library pytest dan pyinstaller

III. Landasan Teori

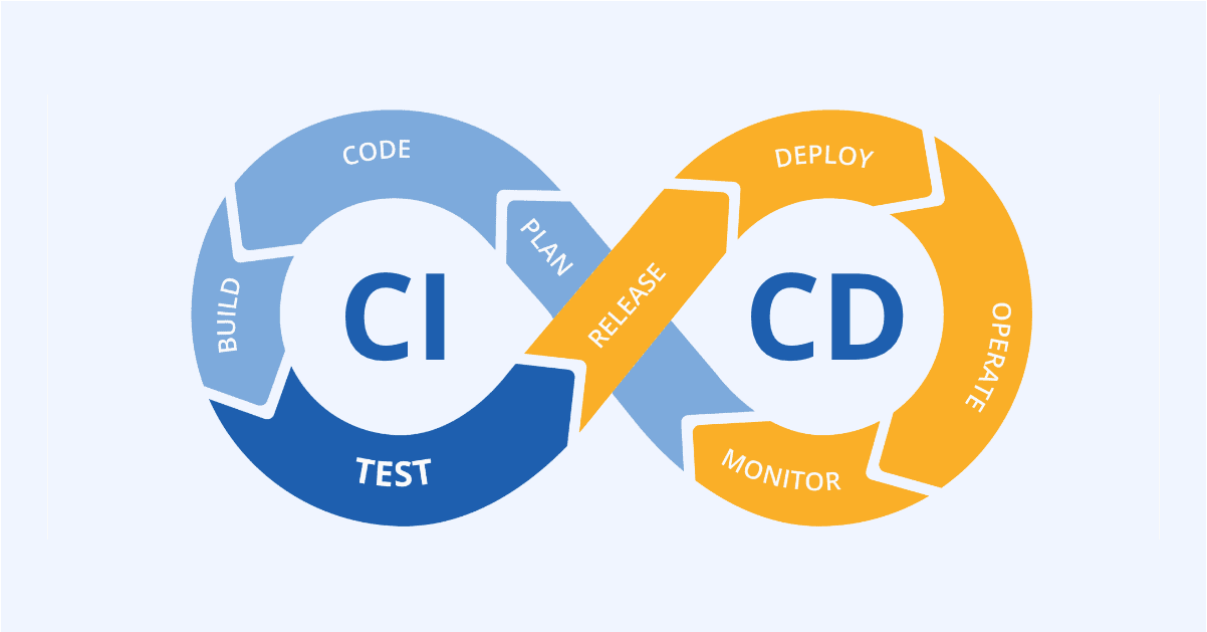
DevOps (gabungan dari kata Development dan Operations) adalah serangkaian praktik, budaya kerja, dan filosofi teknis yang mengintegrasikan tim pengembang perangkat lunak (developer) dengan tim operasional TI (operations). Konsep utama DevOps bertumpu pada kolaborasi dan komunikasi tim, otomatisasi proses, pengiriman berkelanjutan, serta pemantauan dan umpan balik (monitoring and feedback). Tujuan utamanya adalah memperpendek siklus hidup pengembangan sistem (SDLC) sembari memastikan rilis perangkat lunak memiliki kualitas dan keandalan yang tinggi.

Salah satu pilar otomatisasi terpenting dalam DevOps adalah penerapan Continuous Integration (CI) dan Continuous Delivery/Deployment (CD):

1. Continuous Integration (CI): Praktik rekayasa di mana setiap kali anggota tim mengunggah (push) perubahan kode atau membuat Pull Request ke repositori utama, sistem server akan secara otomatis mengunduh kode tersebut, membangun lingkungan kerja (build), dan menjalankan seluruh skrip pengujian unit (unit testing). Penggunaan CI

memastikan setiap bug atau konflik integrasi antarkomponen MVC dapat terdeteksi sedini mungkin sebelum kode digabungkan.

2. Continuous Delivery / Deployment (CD): Kelanjutan dari proses CI yang mengotomatiskan pengemasan dan pendistribusian perangkat lunak yang telah lolos tahap pengujian ke lingkungan produksi atau rilis siap pakai.



Saat ini, platform repositori modern menyediakan layanan otomasi CI/CD berbasis cloud seperti GitHub Actions dan GitLab CI. Melalui berkas konfigurasi berformat YAML (.yml), pengembang dapat menyusun instruksi alur kerja (workflow) yang akan dieksekusi secara otomatis oleh mesin virtual (runner) milik GitHub setiap kali terjadi perubahan kode.

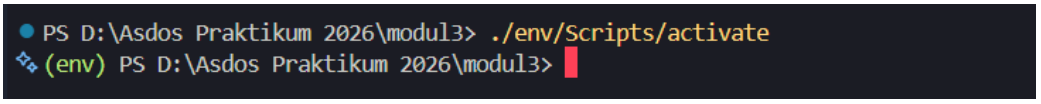
Agar server runner pada GitHub Actions dapat menjalankan kode aplikasi dan skrip pengujian pytest, server tersebut harus menginstal daftar pustaka (packages) yang sama persis dengan yang ada di komputer lokal pengembang. Dalam ekosistem Python, daftar pustaka beserta versinya dibekukan ke dalam sebuah berkas teks standar bernama requirements.txt menggunakan pengelola paket pip. Berkas ini menjamin lingkungan pengembangan bersifat reprodusibel (dapat dibangun ulang dengan hasil yang identik di komputer atau server mana pun).

Pengguna akhir (end-user) seperti petugas perpustakaan umumnya tidak memiliki pengetahuan teknis untuk menginstal interpreter Python, membuat virtual environment, atau menjalankan perintah melalui terminal. Oleh karena itu, tahap akhir dari implementasi perangkat lunak desktop adalah memaketkan (bundling/packaging) seluruh skrip kode sumber (models, views, controllers), interpreter Python, beserta pustaka eksternal (customtkinter, mysql-connector-python) menjadi satu berkas eksekusi mandiri (standalone

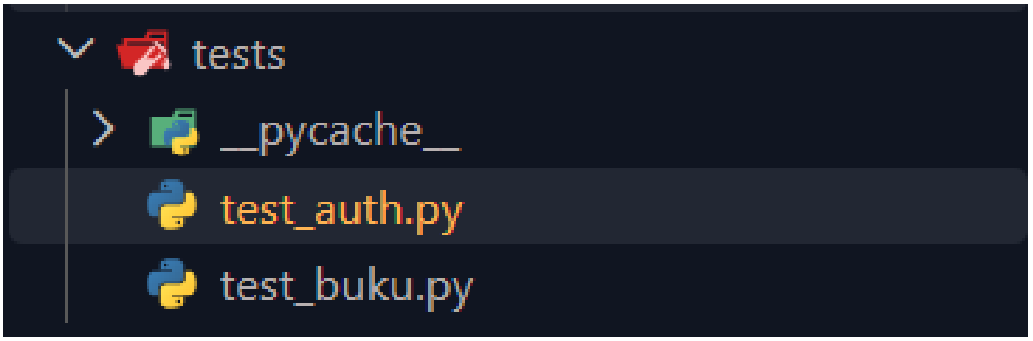
executable berformat .exe pada Windows). Pustaka PyInstaller bekerja dengan menganalisis seluruh pohon dependensi di dalam file titik masuk (main.py), lalu membungkusnya menjadi aplikasi desktop yang dapat langsung dibuka hanya dengan klik ganda (double-click).

IV. Prosedur Kerja

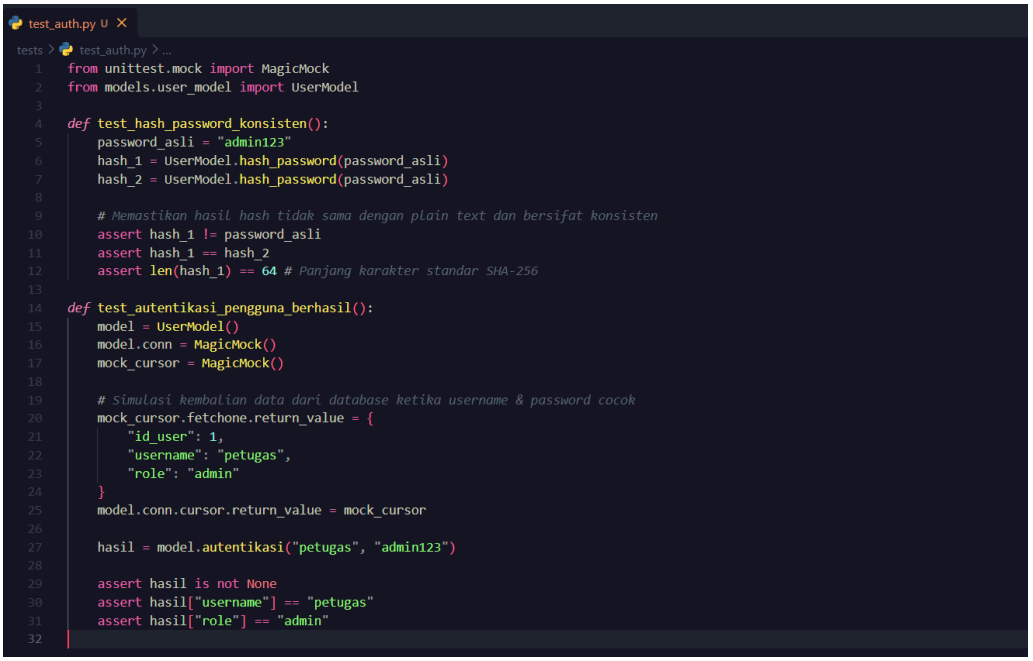
1. Menambahkan Unit Test untuk Fitur Autentikasi
    a. Buka proyek Anda di Visual Studio Code dan pastikan Virtual Environment (env) telah aktif.



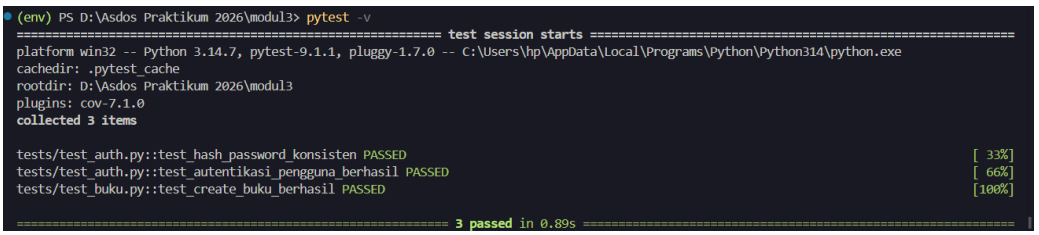
    b. Di dalam folder tests, buat file baru bernama test_auth.py.



    c. Tulis skrip pengujian berikut untuk menguji konsistensi algoritma hashing SHA-256 dan proses autentikasi:



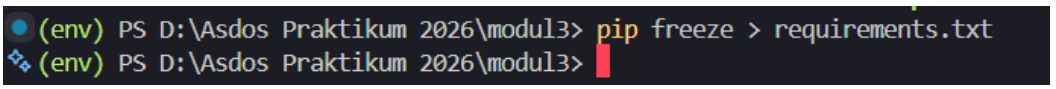
d. Jalankan perintah pytest -v di terminal untuk memastikan seluruh pengujian di test_buku.py dan test_auth.py berstatus PASSED



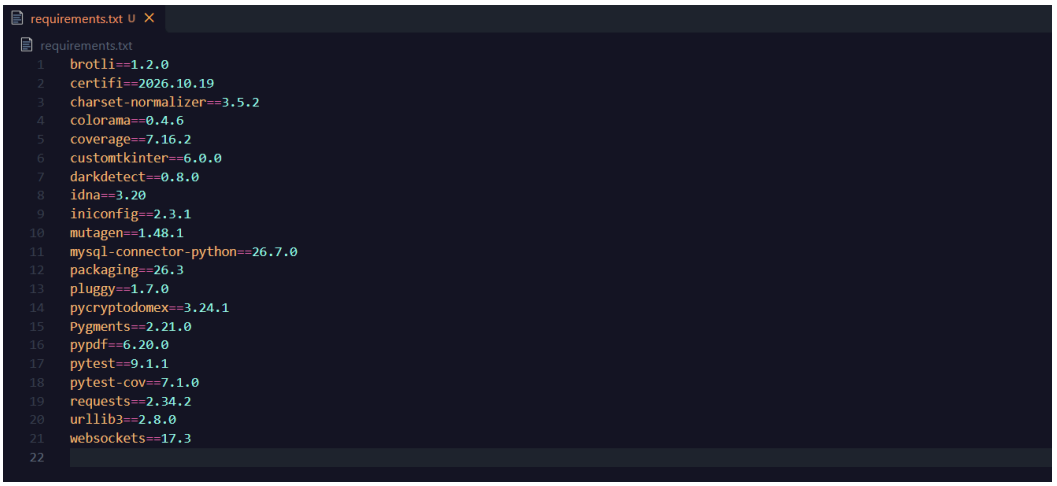
2. Membuat Berkas Daftar Dependensi

a. Masih di dalam terminal dengan env aktif, jalankan perintah berikut untuk menyimpan daftar pustaka proyek ke dalam file requirements.txt:

pip freeze > requirements.txt

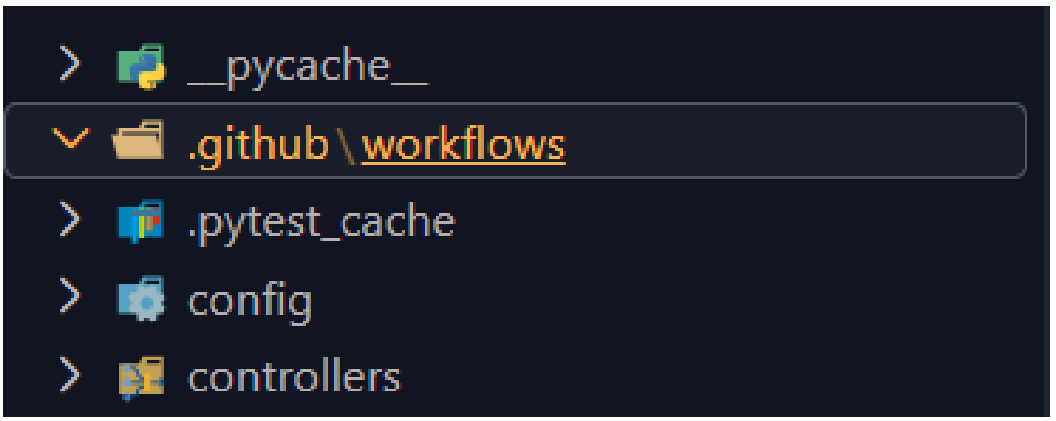


b. Buka file requirements.txt yang baru terbentuk di root direktori proyek Anda, pastikan di dalamnya telah memuat pustaka utama seperti mysql-connector-python, customtkinter, dan pytest.

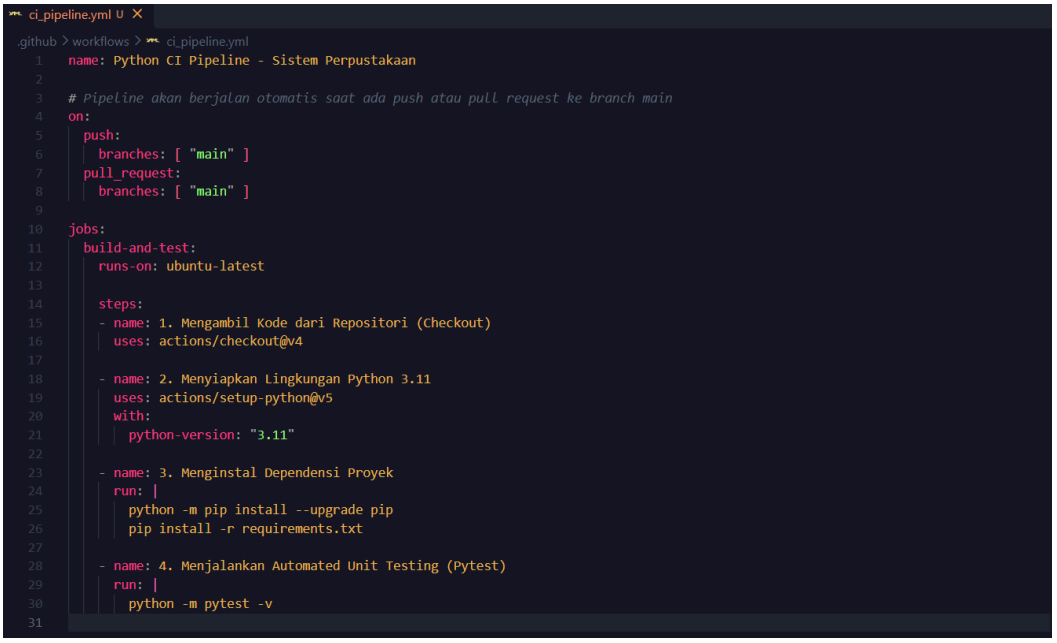


3. Konfigurasi CI Pipeline dengan GitHub Actions

a. Di root direktori proyek Anda, buat folder baru bernama .github dan buat subfolder bernama workflows



b. Di dalam folder .github/workflows, buat file konfigurasi bernama ci_pipeline.yml dan ketikkan konfigurasi pipeline berikut:



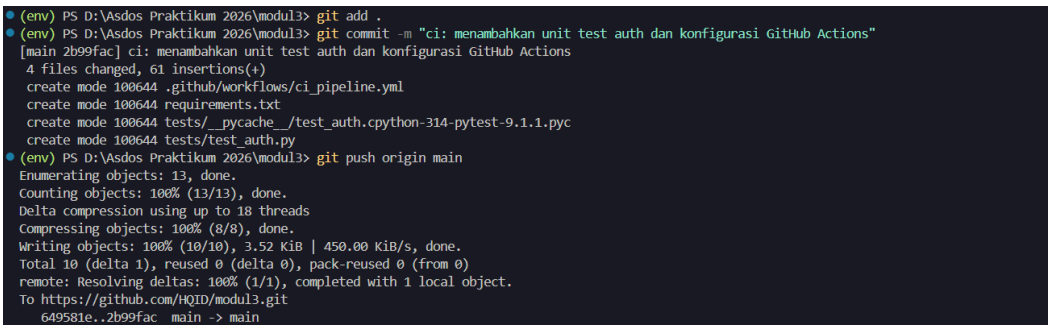
4. Menguji Pipeline CI di GitHub

a. Simpan seluruh perubahan lokal dan kirimkan ke GitHub melalui terminal:

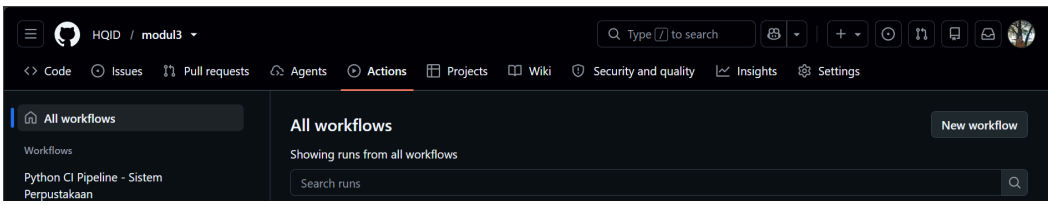
git add .

git commit -m "ci: menambahkan unit test auth dan konfigurasi GitHub Actions"

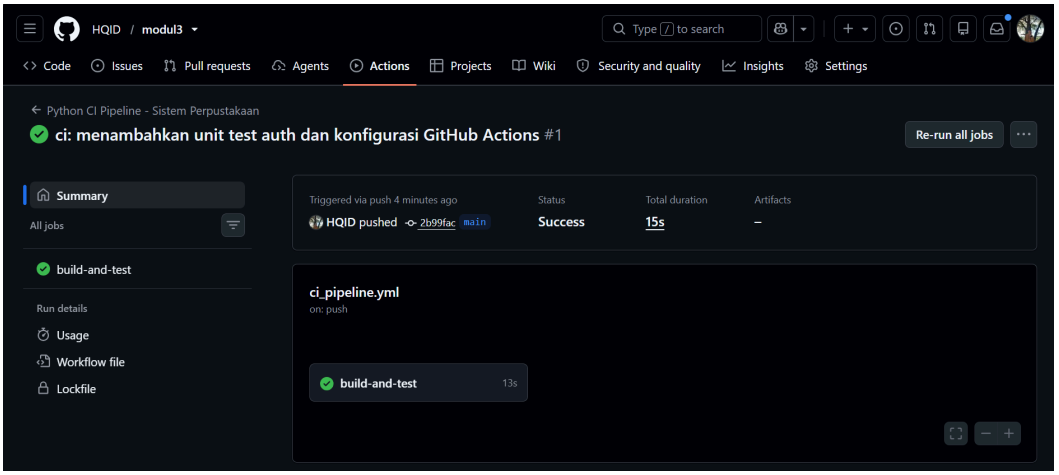
git push origin main



b. Buka repositori proyek Anda di browser, lalu klik tab Actions di bagian atas halaman GitHub.



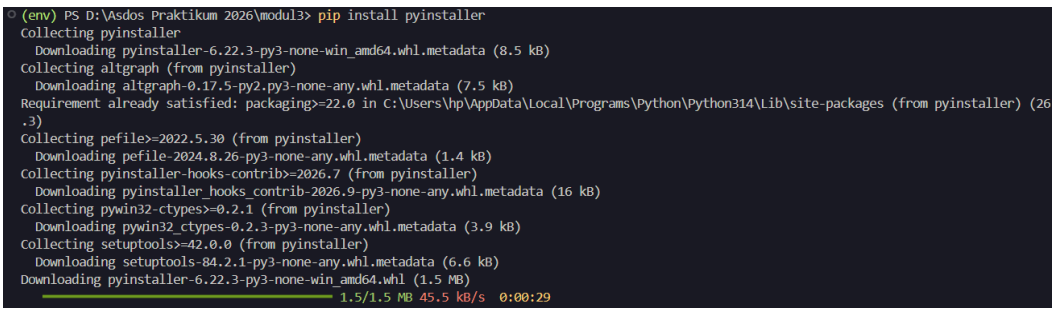
c. Amati proses workflow "Python CI Pipeline - Sistem Perpustakaan" yang sedang berjalan secara otomatis. Tunggu hingga indikator berubah menjadi centang hijau (Success) yang menandakan seluruh pengujian otomatis berhasil dilewati



5. Pemaketan Aplikasi Menjadi Berkas Eksekusi

a. Kembali ke terminal VS Code Anda, instal pustaka pyinstaller:

pip install pyinstaller

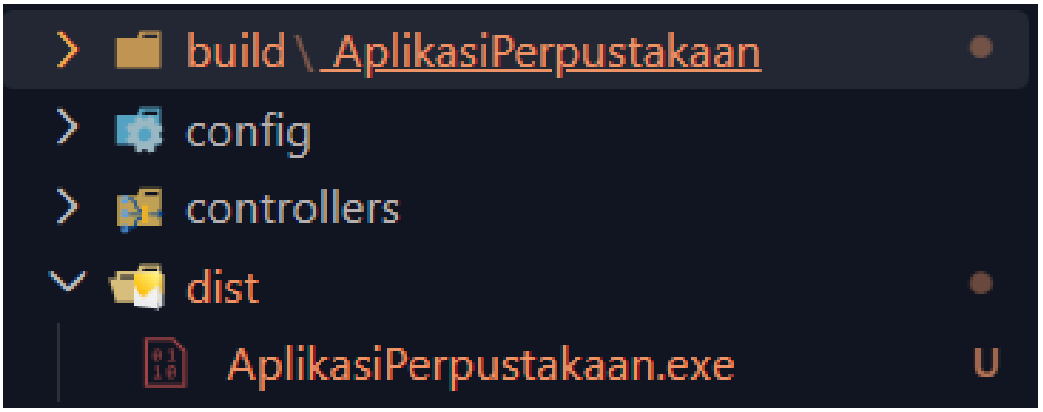


b. Jalankan perintah berikut untuk membungkus proyek menjadi satu berkas .exe tanpa memunculkan layar hitam terminal (console) saat dibuka:

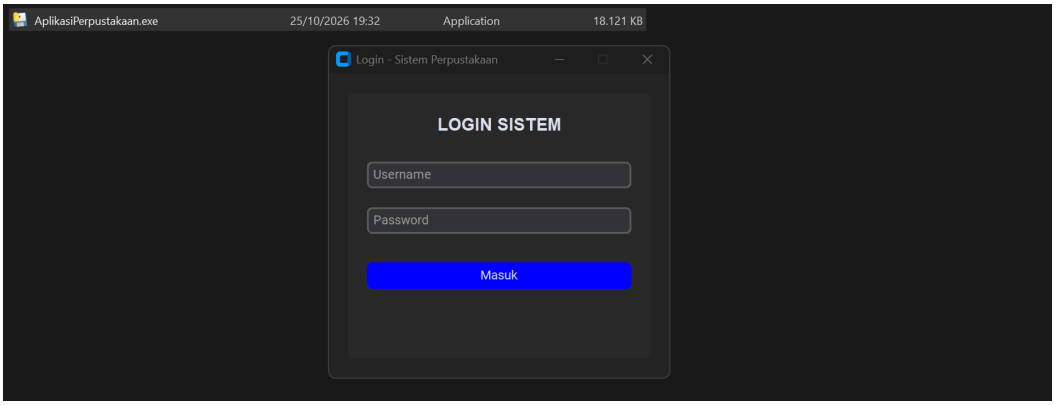
pyinstaller --noconfirm --onefile --windowed --name " AplikasiPerpustakaan" main.py

```
(env) PS D:\Asdos Praktikum 2026\modul3> pyinstaller --noconfirm --onefile --windowed --name " AplikasiPerpustakaan" main.py
528 INFO: PyInstaller: 6.22.3, contrib hooks: 2026.9
529 INFO: Python: 3.14.7
578 INFO: Platform: Windows-11-10.0.26200-SP0
579 INFO: Python environment: C:\Users\hp\AppData\Local\Programs\Python\Python314
580 INFO: wrote D:\Asdos Praktikum 2026\modul3\ AplikasiPerpustakaan.spec
592 INFO: Module search paths (PYTHONPATH):
['C:\\Users\\hp\\AppData\\Local\\Programs\\Python\\Python314\\Scripts\\pyinstaller.exe',
 'C:\\Users\\hp\\AppData\\Local\\Programs\\Python\\Python314\\python314.zip',
 'C:\\Users\\hp\\AppData\\Local\\Programs\\Python\\Python314\\DLLs',
 'C:\\Users\\hp\\AppData\\Local\\Programs\\Python\\Python314\\Lib',
 'C:\\Users\\hp\\AppData\\Local\\Programs\\Python\\Python314',
 'C:\\Users\\hp\\AppData\\Local\\Programs\\Python\\Python314\\Lib\\site-packages',
 'D:\\Asdos Praktikum 2026\\modul3']
1967 INFO: checking Analysis
```

c. Tunggu hingga proses kompilasi selesai. PyInstaller akan menghasilkan dua folder baru, yaitu build/ dan dist/.

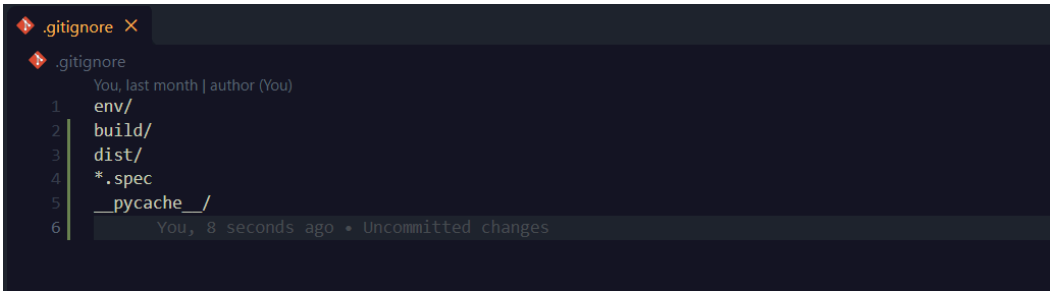


d. Buka folder dist/ melalui Windows Explorer, lalu klik dua kali pada file AplikasiPerpustakaan.exe (pastikan MySQL pada Laragon sedang menyala).



6. Pembersihan dan Penyimpanan Akhir ke Repositori Git

a. Buka file .gitignore dan tambahkan baris berikut di bawah env/:



b. Simpan pembaruan .gitignore dan requirements.txt ke GitHub:

git add .gitignore requirements.txt

git commit -m "chore: update gitignore untuk mengabaikan hasil build pyinstaller"

git push origin main

```
(env) PS D:\Asdos Praktikum 2026\modul3> git add .gitignore requirements.txt
(env) PS D:\Asdos Praktikum 2026\modul3> git commit -m "chore: update gitignore untuk mengabaikan hasil build pyinstaller"
[main 1feb93b] chore: update gitignore untuk mengabaikan hasil build pyinstaller
 1 file changed, 5 insertions(+), 1 deletion(-)
(env) PS D:\Asdos Praktikum 2026\modul3> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 18 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 336 bytes | 336.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/HQID/modul3.git
   2b99fac..1feb93b  main -> main
```

V. Hasil dan Pembahasan

1. Tampilan tab Actions di GitHub
2. Tampilan isi folder dist/ yang menampilkan file AplikasiPerpustakaan.exe
3. Tampilan hasil dijalankannya aplikasi .exe

TUGAS

1. Integrasi Linter pada CI Pipeline: Instal pustaka pengecek kualitas penulisan kode flake8 (pip install flake8), perbarui requirements.txt, lalu tambahkan satu langkah (step) baru di dalam file .github/workflows/ci_pipeline.yml untuk menjalankan perintah:

   flake8 models/ controllers/ --count --select=E9,F63,F7,F82 --show-source --statistics

   Tepat sebelum langkah pengujian pytest. Lakukan push dan lampirkan screenshot bahwa pipeline tetap berstatus hijau (passing)
2. Penyusunan Dokumentasi Teknis (README.md): buatlah sebuah file bernama README.md di root direktori proyek Anda menggunakan format Markdown. Tuliskan di dalamnya: Deskripsi singkat aplikasi, Struktur Direktori MVC, Panduan Instalasi & Konfigurasi Database MySQL, serta Cara Menjalankan Aplikasi.
3. Rilis Produk (GitHub Releases): Unggah file AplikasiPerpustakaan.exe yang ada di dalam folder dist/ ke menu Releases (Buat Release versi v1.0.0) pada halaman repositori GitHub Anda agar pengguna dapat mengunduh installer aplikasi secara langsung. Lampirkan screenshot halaman Release v1.0.0